

PeopleOps HRMS

Academic Software Engineering Report

Human Resource Management System

Laravel, Supabase PostgreSQL, Render, and Tailwind CSS

Prepared as a structured project report covering architecture, database design, implementation, deployment, workflows, results, and future improvements.

PO HRMS

Project: PeopleOps HRMS

Generated: May 2026

Environment: Laravel 10 application deployed on Render with Supabase PostgreSQL

Abstract

PeopleOps HRMS is a web-based human resource management system designed to centralize employee records, role-based access control, departments, attendance, leave approvals, payroll processing, and internal announcements. The software replaces fragmented manual HR operations with an integrated workflow built on Laravel and PostgreSQL. The system follows a modular MVC design, uses authenticated sessions, applies permission-based middleware, and exposes administrative screens for HR teams and managers. This report documents the design and implementation of the software as an academic case study, including requirements, architecture, database design, backend logic, deployment practices, screenshots, outcomes, challenges, and future scope.

Keywords

HRMS, Laravel, PostgreSQL, Supabase, Render, role-based access control, payroll, attendance, leave management, MVC architecture.

Table of Contents

1. Introduction	4
2. Problem Statement	5
3. Existing Problems and Literature Review	6
4. System Architecture	7
5. Database Design	8
6. Backend and API Design	9
7. Technologies Used	10
8. Implementation	11
9. Screenshots and Workflow	12
10. Results and Outcomes	13
11. Challenges and Limitations	14
12. Future Improvements	15
13. Conclusion	16

1. Introduction

Human resource management is a core administrative function in every organization. It involves recruitment records, departmental assignment, attendance tracking, leave management, payroll processing, and internal communication. When these activities are maintained in disconnected spreadsheets, paper registers, and informal messages, HR teams face slow reporting, inconsistent data, and reduced accountability. PeopleOps HRMS addresses this gap by offering a centralized digital platform for operational HR workflows.

The application provides a login-protected workspace where authorized users can manage employees, departments, attendance records, leave requests, payroll entries, announcements, and roles. The design emphasizes a clean dashboard, searchable records, consistent forms, and permissions that can be customized by role. The implementation is suitable for small to mid-sized organizations that need a practical HRMS without excessive setup complexity.

Project Objectives

- Create a centralized HRMS application for employee and people operations data.
- Provide role and permission controls for HR, payroll, employee, and administrative functions.
- Track attendance, leave requests, payroll status, departments, designations, and announcements.
- Deploy the application using Docker on Render while storing production data in Supabase PostgreSQL.

2. Problem Statement

Small teams often begin HR operations with spreadsheets and manual approvals. As the organization grows, the same approach becomes fragile: employee records are duplicated, leave requests are lost, salary calculations become hard to audit, and managers cannot view reliable real-time information. The primary problem is the lack of a unified, permission-aware, and web-accessible HR platform.

Core Problems Identified

- Employee master data is difficult to maintain consistently across HR, finance, and management teams.
- Attendance and leave data are often disconnected, making absence reporting slow and error-prone.
- Payroll entries need traceable status management and department-level employee context.
- Administrative access must be restricted so users only see and modify the workflows relevant to their roles.
- Deployment should be simple enough for cloud hosting while supporting a reliable managed database.

3. Existing Problems and Literature Review

Traditional HR information systems have evolved from record-keeping systems into integrated human capital platforms. Common HRMS literature emphasizes data centralization, self-service, role-based authorization, workflow automation, and reporting accuracy. Modern web frameworks enable these features at lower cost by combining MVC architecture, relational databases, reusable UI components, and cloud deployment.

Observed Gaps in Lightweight HR Systems

- Spreadsheet systems are flexible but weak in validation, concurrency, history, and authorization.
- Generic project templates provide authentication but usually lack HR-specific data models.
- Many payroll and attendance tools are isolated products and do not share a common employee profile.
- Cloud deployment commonly fails when environment variables, storage permissions, HTTPS proxy settings, or database connectivity are not configured correctly.

4. System Architecture

The system follows a layered architecture. Users interact with responsive Blade templates styled with Tailwind CSS and compiled by Vite. Requests pass through Render, Nginx, PHP-FPM, Laravel routes, authentication middleware, and permission middleware before reaching controllers. Controllers coordinate models and database operations. Supabase provides managed PostgreSQL storage using a transaction pooler suitable for IPv4 hosting environments such as Render.

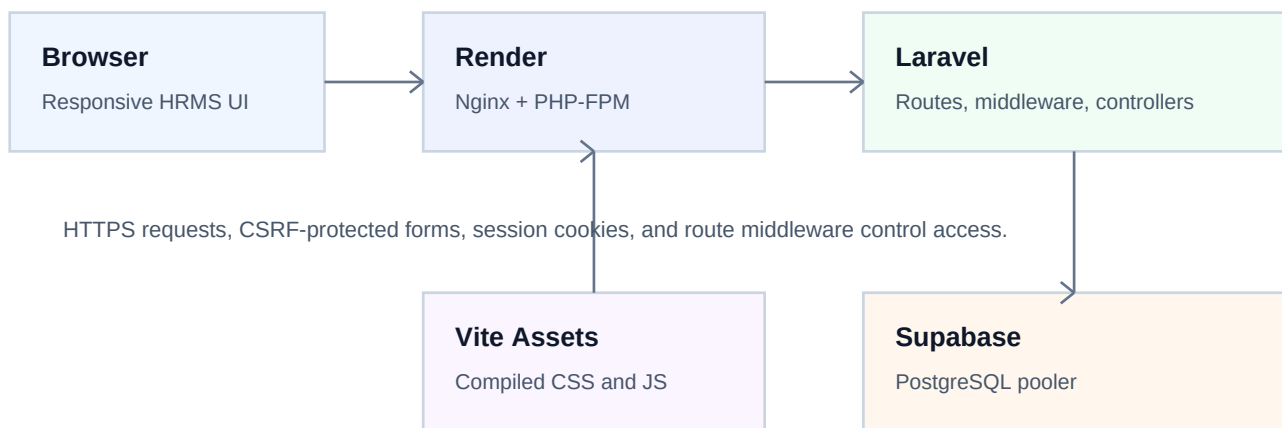


Figure 1. High-level deployment and request architecture.

4.1 MVC and Request Flow

Laravel implements the Model-View-Controller pattern. Routes map URLs to controller methods, controllers validate and process user actions, Eloquent models represent tables, and Blade views render user interfaces. Middleware adds cross-cutting behavior such as authentication, CSRF protection, and permission checks.

1**Browser form submits POST /login**

CSRF token and session cookie are sent.

2**Route resolves controller action**

web.php maps login and HRMS routes.

3**Middleware validates access**

auth and permission middleware protect modules.

4**Controller performs business logic**

Attendance, leave, payroll, and employee controllers update models.

5**Model persists data**

Eloquent writes to PostgreSQL through pgsql connection.

6**Blade renders response**

Views return the next screen or redirect.

5. Database Design

The database is normalized around a users table and HR-specific tables. Roles store permission slugs as JSON. Employee profiles connect users to departments and designations. Attendance records store daily clock and status information. Leave requests connect users to leave types and approval metadata. Payroll records store pay period, gross components, deductions, tax, net pay, and payment status.

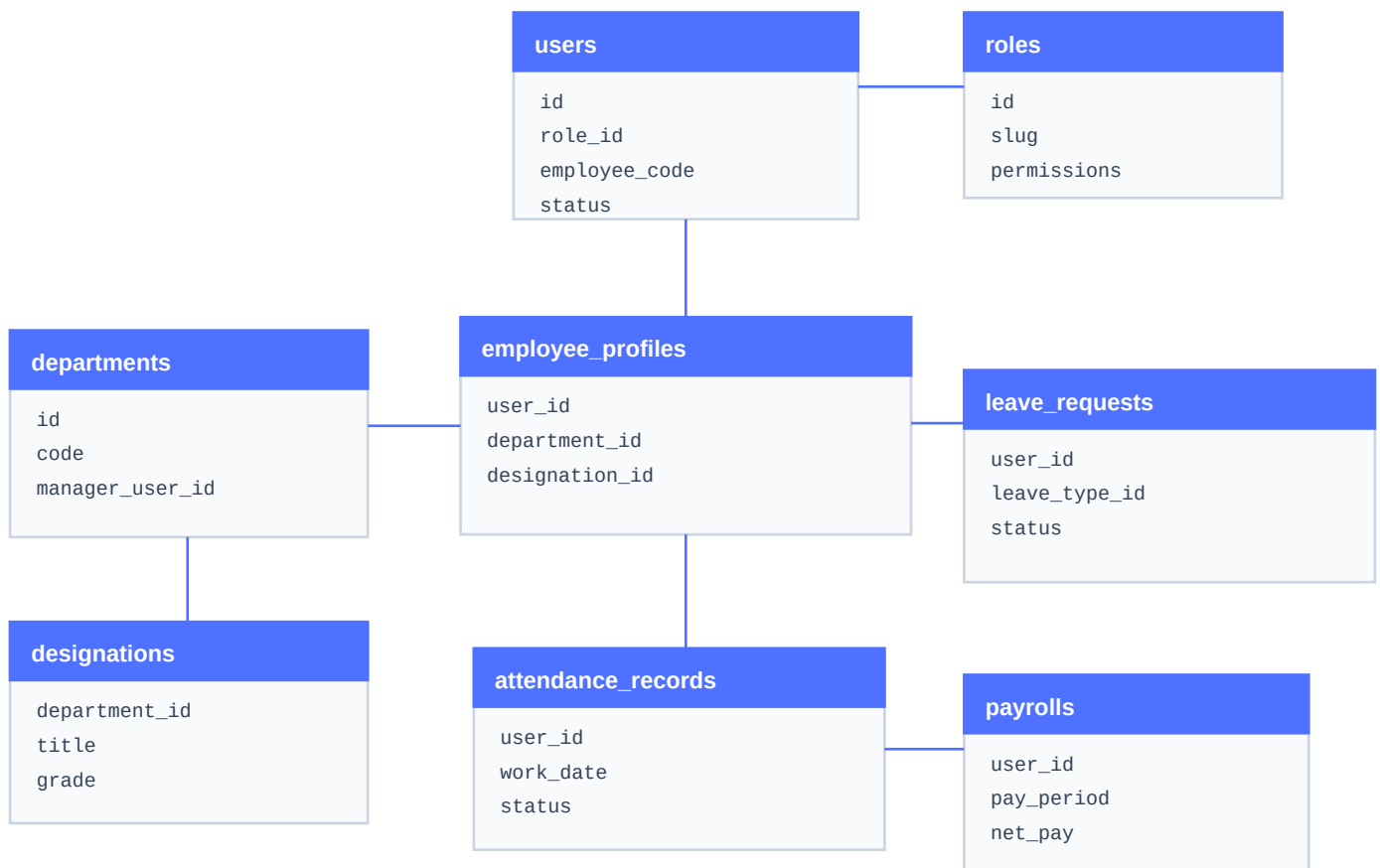


Figure 2. Simplified entity relationship diagram for the HRMS schema.

5.1 Table Summary

`users`

Authentication identity, employee code, account status, contact details, and role assignment.

`roles`

Readable role names, slugs, descriptions, status, and JSON permission lists.

`departments`

Organization units with codes, descriptions, status, and optional manager references.

`designations`

Job titles grouped by department and grade.

`employee_profiles`

Employment metadata including manager, location, salary, address, and bank details.

`attendance_records`

Daily work records with clock-in, clock-out, status, and notes.

`leave_requests`

Employee leave periods, type, reason, status, approval, and rejection notes.

`payrolls`

Pay period records for basic salary, allowances, deductions, tax, net pay, and payment status.

`announcements`

Company-wide or department-specific communication items.

6. Backend and API Design

The application is primarily server-rendered using Laravel routes and Blade views. It does not depend on a separate SPA API for normal operation. Backend design focuses on route groups, named routes, controller methods, request validation, Eloquent relationships, and middleware. The web guard handles authentication, and custom permission middleware checks role permissions before allowing module access.

Code excerpt

```
Route::middleware('auth')->group(function () {
    Route::get('/dashboard', DashboardController::class)
        ->middleware('permission:dashboard.view')
        ->name('dashboard');

    Route::get('/employees', [EmployeeController::class, 'index'])
        ->middleware('permission:employees.view')
        ->name('employees.index');
});
```

The route naming convention makes links stable across views. Permissions are attached to routes rather than hidden only in the UI, which prevents unauthorized users from accessing protected actions by typing URLs manually.

6.1 Role and Permission Logic

The permission system converts traditional short role codes into readable permission slugs such as `dashboard.view`, `employees.manage`, `leave.manage`, `payroll.view`, and `roles.manage`. This makes authorization easier to audit and extend. Each user belongs to a role. The role stores a list of permissions, and middleware checks the current route requirement against the authenticated user.

Code excerpt

```
public function hasPermission(string $permission): bool
{
    if (! $this->role || ! $this->role->status) {
        return false;
    }

    return in_array($permission, $this->role->permissions ?? [], true);
}
```

- Super Admin receives broad access to all HRMS modules.
- HR Manager can manage people records, departments, attendance, leave, announcements, and roles.
- Payroll Specialist can view and process salary records without unnecessary HR administration access.
- Employee access is limited to self-service workflows such as attendance, leave, announcements, and payslips.

7. Technologies Used

Laravel 10

PHP framework used for routing, MVC structure, sessions, middleware, Blade templates, Eloquent ORM, and migrations.

PostgreSQL / Supabase

Managed relational database used for production storage through the Supabase transaction pooler.

Render

Docker-based hosting platform used for deploying the web service.

Nginx and PHP-FPM

Container runtime stack that serves public assets and executes Laravel requests.

Tailwind CSS and Vite

Frontend styling and asset build pipeline for responsive UI.

Docker

Defines a reproducible deployment image with PHP extensions, Node build, Composer dependencies, and startup script.

8. Implementation

Implementation began by converting the original Laravel application into a dedicated HRMS. New models, controllers, views, migrations, seeders, and middleware were introduced. The database was rebuilt around HRMS concepts. The user interface was redesigned to open directly into a login and dashboard workflow rather than the default Laravel welcome page. Demo data was seeded to provide realistic departments, employees, attendance, leave, payroll, and announcement records.

Implemented Modules

- Dashboard with employee, department, attendance, and leave summaries.
- Employee master records with department, designation, role, status, and joining date.
- Department and designation management.
- Attendance list, clock-in/clock-out actions, status updates, and notes.
- Leave request submission, approval, rejection, and status filtering.
- Payroll entry and register with processed/paid statuses.
- Announcements and role-permission administration.

8.1 Deployment Implementation

Deployment required configuring the Docker image for PHP-FPM, Nginx, Composer, Node, and required PHP extensions. Because the production database uses Supabase PostgreSQL, the pdo_pgsql extension is included. The startup script prepares Laravel writable folders, clears and caches configuration, and starts PHP-FPM with Nginx. Runtime logs are sent to stderr, which is the expected logging channel for Render.

Code excerpt

```
RUN docker-php-ext-install \
    pdo_mysql \
    pdo_pgsql \
    mbstring \
    zip \
    gd \
    bcmath

if [ "${RUN_MIGRATIONS:-false}" = "true" ]; then
    php artisan migrate --force
fi
```

A key deployment lesson was that Supabase direct database hosts may not be IPv4 compatible. Render should use the Supabase transaction pooler, typically on port 6543, with DB_SSLMODE=require.

9. Screenshots and Workflow

The following figures summarize the uploaded screenshots and user workflow. The PDF uses clean vector reproductions of the screens so that the figures remain readable in print.

PeopleOps HRMS - Login

PO

PeopleOps HRMS
Email

Password

LOG IN

Figure 3. Login screen workflow based on the uploaded login screenshot.

PeopleOps HRMS - Dashboard

Dashboard
Employees
Departments
Attendance
Leave
Payroll

HRMS Dashboard

Employees
10

Departments
5

Present
11

Leaves
2

Employee	Department	Role	Status

Announcements

Figure 4. Dashboard workflow with summary cards, recent employees, announcements, and pending leave.

9.1 Employee and Leave Workflows

PeopleOps HRMS - Employees

Dashboard

Employees

Departments

Attendance

Leave

Payroll

Employees

Search employees

Department

Employee	Department	Designation	Status

Figure 5. Employee list, search, filters, status badges, and edit actions.

PeopleOps HRMS - Leave

Dashboard

Employees

Departments

Attendance

Leave

Payroll

Leave Management

Request Leave

Leave Type

Dates

Employee	Type	Status	Action

Figure 6. Leave request and approval workflow.

PeopleOps HRMS

17

9.2 Workflow Narrative

Authentication and Access

Users begin at the login screen and authenticate through Laravel sessions. After successful login, they are redirected to the dashboard. Role permissions determine which navigation items and routes are usable.

Operational HR Flow

- HR creates departments and designations before assigning employees.
- Employee records are added with role, department, designation, salary, manager, and status details.
- Attendance is recorded daily, with status updates for present, absent, late, and leave cases.
- Employees or HR users submit leave requests; authorized users approve or reject pending requests.
- Payroll entries are generated for a pay period, processed, and then marked as paid.
- Announcements are published to all employees or scoped to a department.

Administrative Flow

Administrators maintain role permission lists through the Roles and Permissions screen. The approach supports evolving access rules without hardcoding every role into controller logic.

10. Results and Outcomes

The completed application provides a working HRMS prototype with realistic seeded data and production deployment support. The main outcome is a coherent HR workflow that integrates employee records, organizational structure, attendance, leave, payroll, announcements, and access control into one Laravel application.

Measured Seed Data Outcome

- 12 users with roles and employee profiles.
- 6 roles with readable permission slugs.
- 5 departments and 16 designations.
- 60 attendance records for realistic daily reporting.
- 4 leave types and 4 leave requests covering pending, approved, and rejected states.
- 24 payroll rows across April and May 2026.
- 3 announcements and 4 asset records.

Functional Outcomes

The application now opens to a branded login page, shows a fully styled dashboard, supports permission-protected HR modules, and can be deployed with Render and Supabase. The UI is appropriate for repeated administrative work rather than a marketing-style landing page.

11. Challenges and Limitations

- Database migration: the original MySQL/MariaDB dump needed to be replaced with Laravel migrations and Supabase-compatible PostgreSQL schema creation.
- PostgreSQL driver: local PHP and the Docker image required pdo_pgsql to connect to Supabase.
- Render networking: Supabase direct hosts may resolve to IPv6, so Render requires the transaction pooler for reliable IPv4 access.
- Storage permissions: Laravel storage and cache directories must be writable at container runtime.
- HTTPS proxy behavior: Render terminates HTTPS before the container, so Laravel must trust forwarded proxy headers and force secure URLs.
- Current scope: the system is a strong operational prototype but does not yet include biometric attendance, PDF payslips, audit logs, or advanced reporting.

12. Future Improvements

- Add PDF payslip generation and employee payslip download history.
- Introduce audit logs for all administrative changes such as role edits, payroll updates, and leave approvals.
- Add dashboard charts for attendance trends, salary cost, department headcount, and leave utilization.
- Add email notifications for leave status, payroll processing, and announcements.
- Create manager self-service workflows for team-level approvals and attendance review.
- Add data export options for CSV, Excel, and month-end payroll reports.
- Improve mobile layouts and add progressive web app behavior for employee self-service.
- Add automated CI checks for migrations, feature tests, build output, and Docker image startup.

13. Conclusion

PeopleOps HRMS demonstrates how a Laravel application can be transformed into a complete human resource management platform. The system provides practical modules for employees, departments, attendance, leave, payroll, announcements, and roles. Its normalized database design, permission middleware, Blade-based interface, and cloud deployment configuration make it suitable as a foundation for a production HRMS.

The project also highlights important deployment lessons: managed PostgreSQL requires the correct driver, container storage must be writable, logs should go to stderr on cloud platforms, and HTTPS proxy handling must be configured explicitly. With future improvements such as audit trails, analytics, notifications, and payslip exports, the system can evolve from a working prototype into a richer HR operations product.

Final Statement

The developed software meets its core objective: it turns fragmented HR activities into a structured, role-aware, database-backed web application that can be accessed securely from a cloud deployment.